

The background features a dark blue gradient with large, overlapping, curved shapes in shades of orange, pink, and purple on the right side, creating a modern, abstract design.

AWS re:Invent

NOV. 27 – DEC. 1, 2023 | LAS VEGAS, NV

OPN307-R

Beyond caching: Advanced design patterns with Redis

Madelyn Olson

(she/they)

Principal Software Engineer

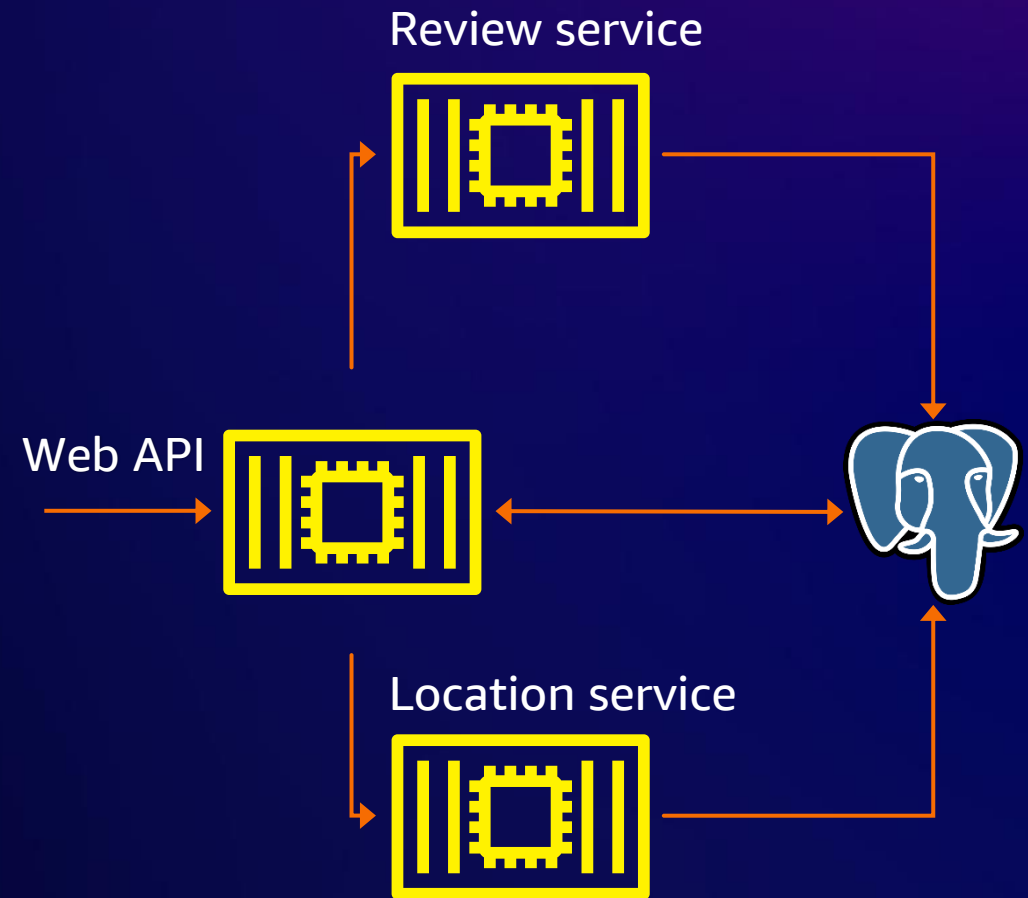
AWS



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Starting with a slow service

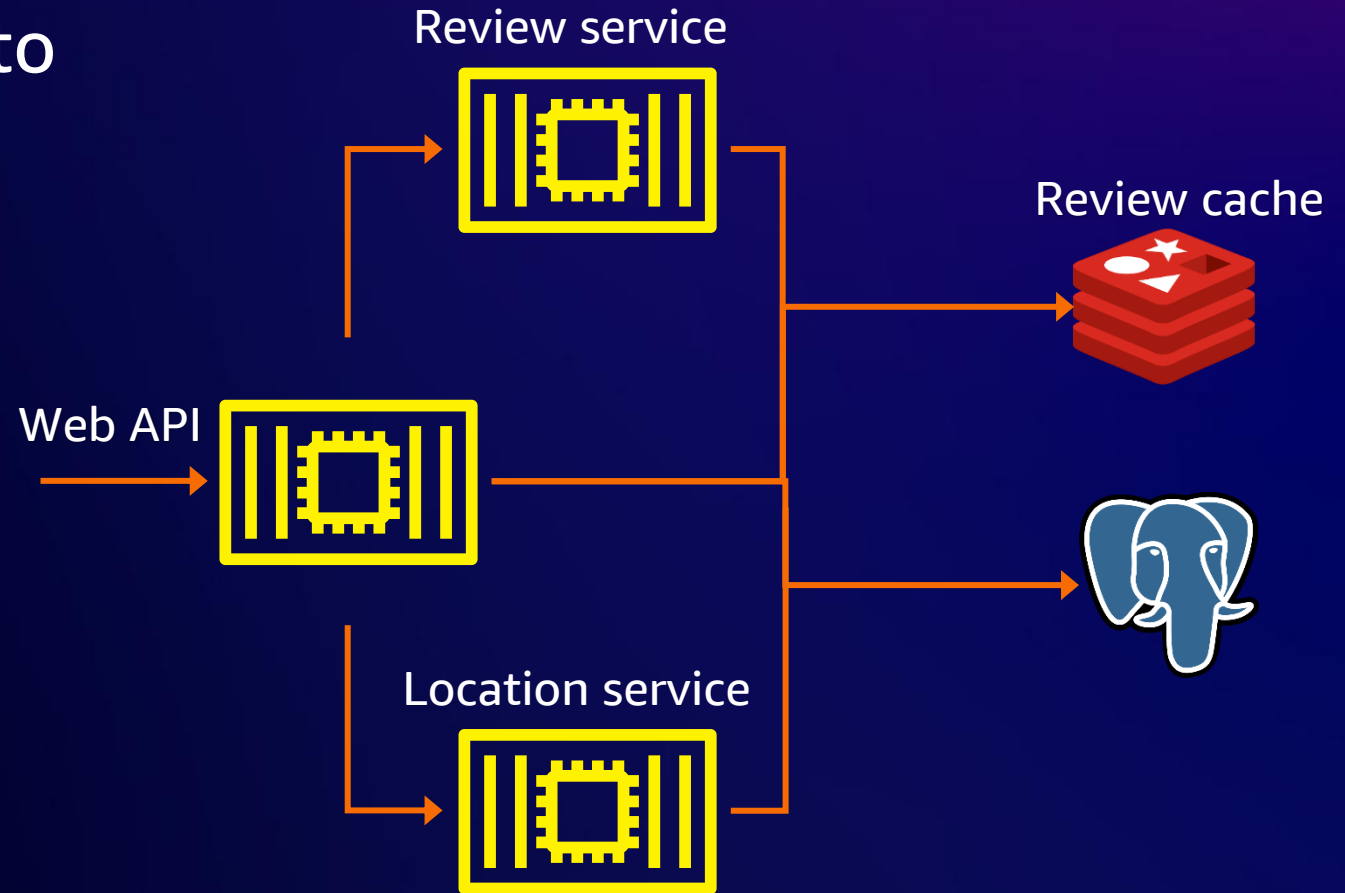
- Three main workflows
 - Search reviews for places
 - Find nearby highly rated places
 - Accept new reviews
- All interactions are backed by a primary database



Making things a bit faster

- Application is slow, add Redis to make it **fast**
- Cache expensive location aggregation searches

“Redis is just a cache” right?
Is it “just a cache”?



“Multitool of distributed systems”

- Simple APIs to access structured data
- Optimized for performance
- Flexible consistency and durability guarantees



Defining how to access the data

`locations:{12853}:name` -> “Hot Cashew Yoga”

`locations:{12853}:location` -> “124 Terry Ave”

`locations:{12853}:data` -> {“name”: “Hot Cashew Yoga”,
“location”: 124 Terry Ave”}

- Understand the access pattern, identify key lookups
- Keys are not ordered, but often follow a **directory-like structure**
- In cluster mode, values are assigned to nodes based on their key name or the **{hashtag}**
- **Collections data structures** can be used to associate data together

Composing access patterns together

locations:{12853}:data -> ...

locations:{Las Vegas}:featured_places -> ["12853"] (TTL 1800)

locations:{Las Vegas}:recent_reviews -> ["re:Invent is so much fun!", "I love open source"]

- Solve complex use cases by **composing multiple data structures**
- Transactions and scripts can execute multiple commands together
- Keys can be used as **indexes for other values**
- **TTLs** provide a time bound of data validity

Pick the datatype for your access pattern

String

```
"Hi there! My name is Madelyn!"
```

Hash

```
{ A:"hello", B:"wonderful", C:"world" }
```

Set

```
{ A, C, E, D, B }
```

Sorted set

```
{ A:-1.0, B:2.5, C:17.9, D:25.0, E:50.0 }
```

Geospatial

```
{ A:(32.1,34.7), B:(51.5,0.12) }
```

List

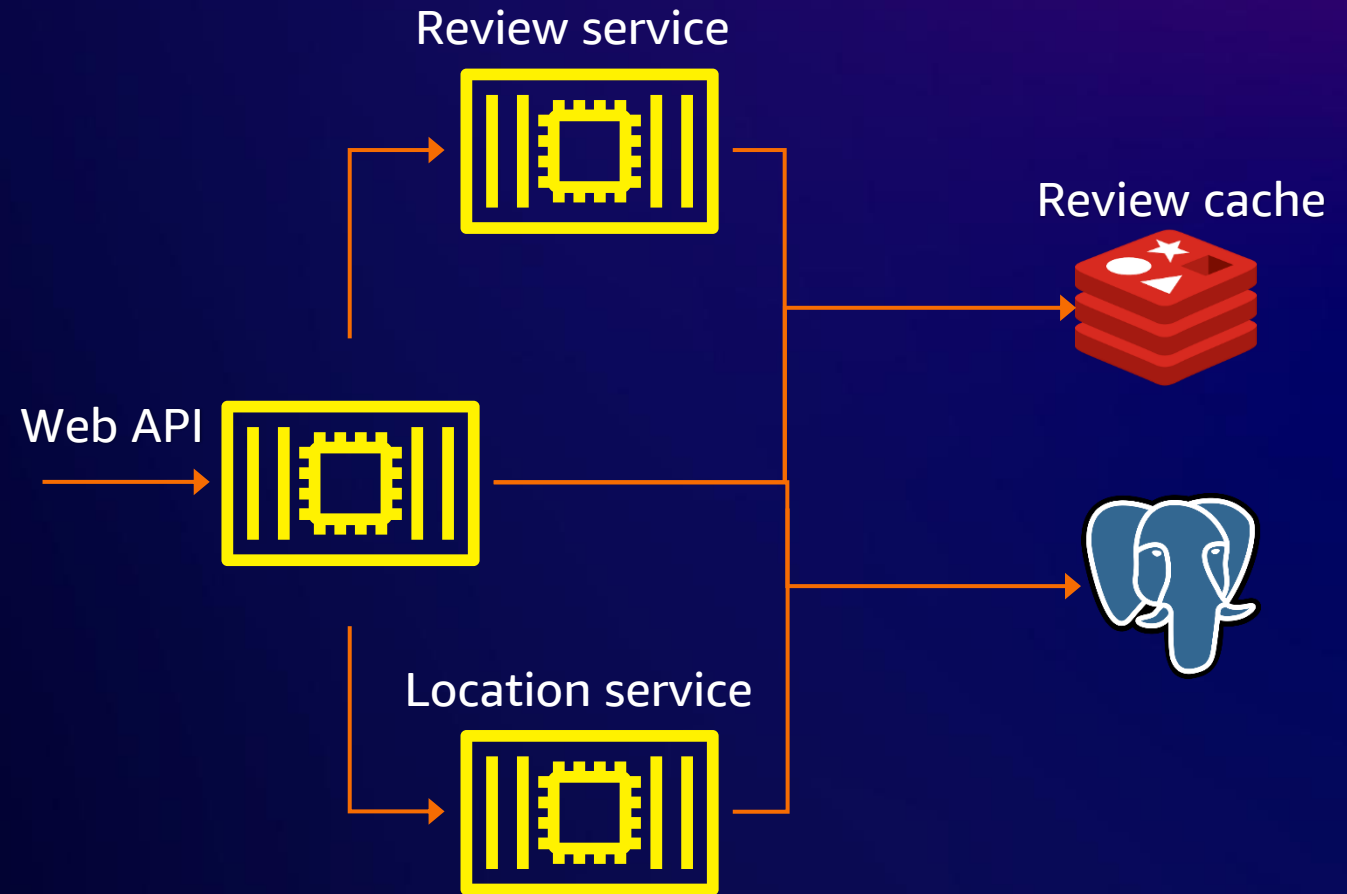
```
[ A, B, C, D, E ]
```

Stream

```
[ (1, {A: "hello"}), (2, {B: "world"}) ]
```

Applying learnings to use cases

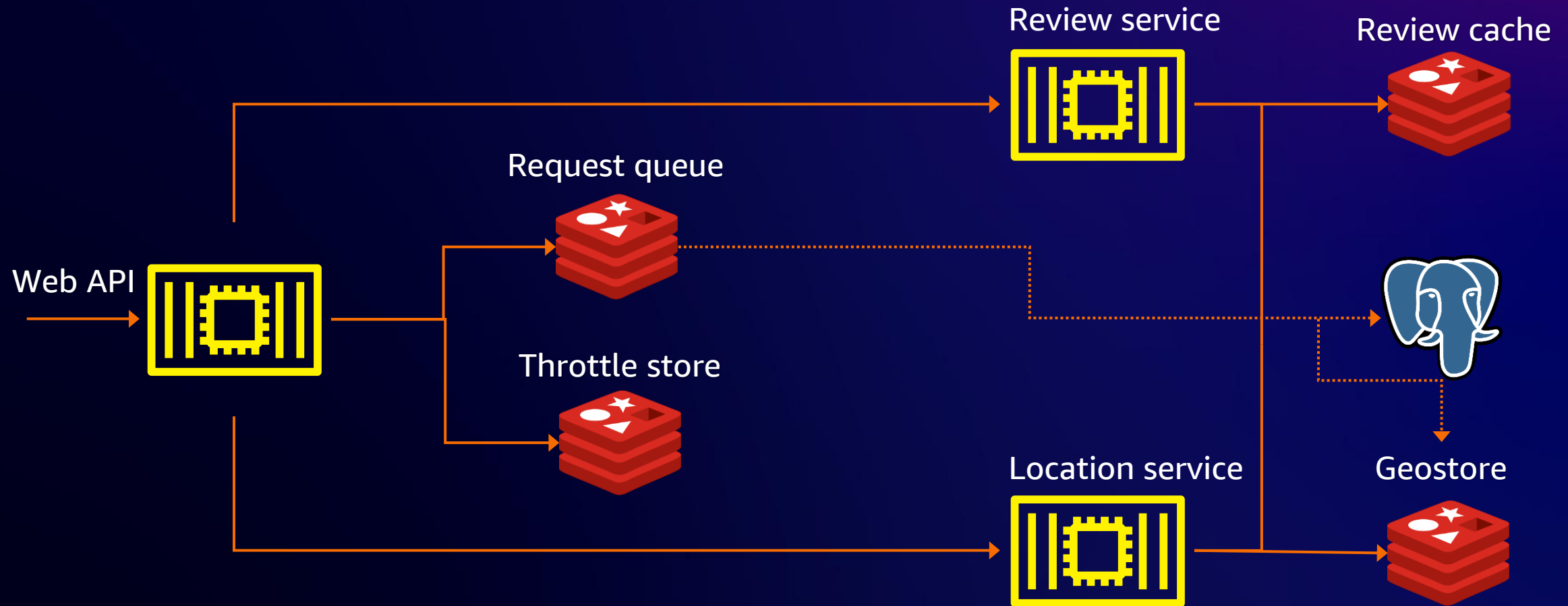
- Handle rate limiting of requests
- Use a message queue for review ingestion
- Improve location performance with a geospatial index



Whiteboard



Possible Redis uses



Key concepts

- Redis is a distributed API to access shared data
- Work back from the access pattern
- Compose data structures together for consistent performance

Thank you!

Madelyn Olson

@reconditerose on X



Please complete the session
survey in the mobile app